Reason. Plan. Act. Physical AI Workshop

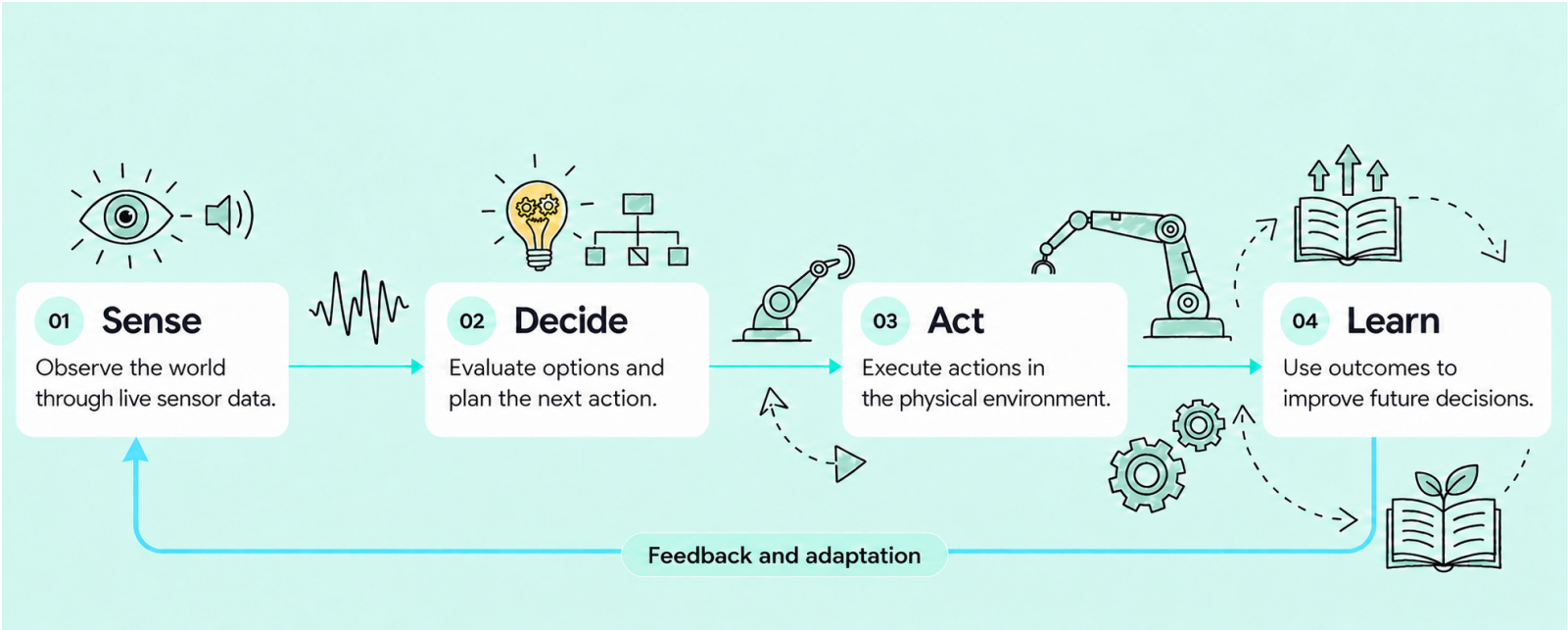
Lecture 4a: Integrating Learning and Robotics

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

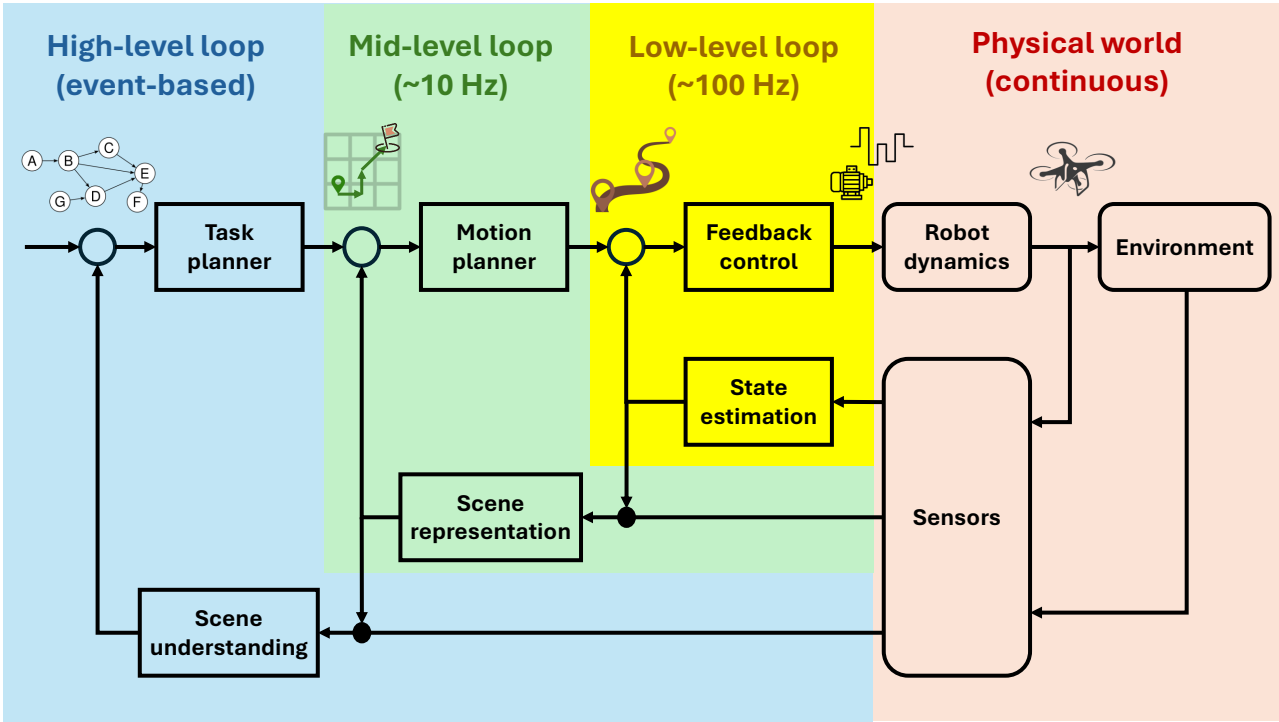
The Continuous Closed Loop

Continuous interaction between intelligence and the physical world.



’Pick up the tennis ball’
’Pick up an apple’
’I am hungry’

Layered Decision-Making



Why Supervised Learning in Robotics?

Robots operate in environments that are difficult to **model explicitly** and encounter tasks that are difficult to **program by hand**.

Learn from examples: Supervised learning maps sensor observations to useful predictions; imitation learning maps demonstrations to actions.

Why Supervised Learning in Robotics?

- **Handle rich sensory inputs.**
  Learned representations help extract objects, spatial relationships, and task-relevant information from images.
- **Reuse knowledge across tasks.**
  Foundation models provide representations learned from large, diverse datasets, reducing the need to start from scratch.
- **Specify goals through language.**
  Vision–language models connect human instructions to what the robot observes.

The promise: less hand engineering and more adaptable robots.

Supervised Learning

Learning from Data

A central goal of machine learning is to automatically discover patterns from data. Instead of manually programming rules, we provide the system with data and allow it to learn a model.

Examples of tasks that can be learned from data:

- Recognizing objects in images
- Predicting house prices
- Detecting anomalies in sensor readings
- Classifying medical images

Different learning paradigms exist depending on the type of data and feedback available.

Supervised Learning

Supervised learning is a paradigm in machine learning where a model is trained using labeled examples.

It is one of the most common learning paradigms (and the core of most foundation models).

Supervised learning answers the question:

Assuming an unknown true distribution $\mathcal{D}$, how can we learn to make predictions from labeled examples sampled from $\mathcal{D}$?

Supervised Learning Components

A **distribution** $\mathcal{D}$, from which samples are drawn i.i.d. (independent and identically distributed)

A **sample**, or labeled example (x_i, y_i) is a pair consisting of:

- an **input** (feature vector) x_i
- the corresponding **label** or target value y_i

Supervised Learning Components

A **distribution** $\mathcal{D}$, from which samples are drawn i.i.d. (independent and identically distributed)

A **sample**, or labeled example (x_i, y_i) is a pair consisting of:

- an **input** (feature vector) x_i
- the corresponding **label** or target value y_i

A **sample set**: $S = \{(x_i, y_i)\}_{i=1}^{m}$ is used to learn a **predictor** $h(x) \in H$ from a **hypothesis class** H that generalizes well with respect to the underlying distribution $\mathcal{D}$, i.e., predicts the correct label for new inputs $(x, y) \sim \mathcal{D}$ with high probability.

Supervised Learning Components

A **distribution** $\mathcal{D}$, from which samples are drawn i.i.d. (independent and identically distributed)

A **sample**, or labeled example (x_i, y_i) is a pair consisting of:

- an **input** (feature vector) x_i
- the corresponding **label** or target value y_i

A **sample set**: $S = \{(x_i, y_i)\}_{i=1}^{m}$ is used to learn a **predictor** $h(x) \in H$ from a **hypothesis class** H that generalizes well with respect to the underlying distribution $\mathcal{D}$, i.e., predicts the correct label for new inputs $(x, y) \sim \mathcal{D}$ with high probability.

When the label y takes values in a **discrete set** (e.g., $\{0, 1\}$ or a set of classes), the task is called **classification**.

When the label y takes values in a **continuous space** (e.g., $y \in \mathbb{R}$), the task is called **regression**.

Supervised Learning Setup: Example

A sample includes information x_i about a person (age, gender, city of birth, ...) and her height y_i.

$x_1 = \{35, F, NY\}, \quad y_1 = 1.58$

$x_2 = \{5, M, TLV\}, \quad y_2 = 0.7$

We seek a predictor $h(x) \in H$ among a hypothesis class H, e.g., linear functions, decision trees, neural networks.

For example, we may seek a linear model

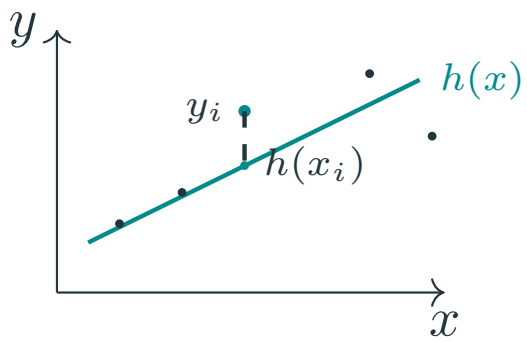
$$H = \left\{ h_w(x) = w^\top x + b \;\middle|\; w \in \mathbb{R}^d,\ b \in \mathbb{R} \right\}$$

where w are the weights and b is the bias.

Loss and Risk

In supervised learning, we aim to **learn a hypothesis that minimizes prediction error on unseen data**.

Loss $\ell(h(x), y)$: point-wise error on a **single example**



Expected Risk $\mathcal{R}(h)$: expected loss over the **data distribution**

$$\mathcal{R}(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(h(x), y)]$$

Since we typically do not have access to the true distribution $\mathcal{D}$, we rely instead on a **finite dataset** $(x, y) \sim \mathcal{D}$ to compute the **Empirical Risk** $\hat{\mathcal{R}}(h)$, i.e., the average loss over the **training data**.

Empirical Risk Minimization

Given a sample set $S = \{(x_i, y_i)\}_{i=1}^m$ and a hypothesis class H, we evaluate each hypothesis by its average loss on the training data.

For a hypothesis $h \in H$, the **empirical risk** is

$$\hat{\mathcal{R}}(h) = \frac{1}{m} \sum_{i=1}^{m} \ell(h(x_i), y_i)$$

The learning problem is to find a hypothesis with minimal empirical risk:

$$\hat{h} = \arg\min_{h \in H} \hat{\mathcal{R}}(h) = \arg\min_{h \in H} \frac{1}{m} \sum_{i=1}^{m} \ell(h \in h(x_i), y_i)$$

Interpretation: choose the predictor whose predictions incur the smallest average loss on the observed examples.

This principle is called **Empirical Risk Minimization (ERM)**.

Examples of Loss Functions

Classification

- 0-1 Loss

$$\ell(h(x), y) = \begin{cases} 0 & h(x) = y \\ 1 & h(x) \neq y \end{cases}$$

- **Cross-Entropy Loss** (used with probabilistic classifiers)

$$\ell(h(x), y) = -\log P(y|x)$$

Regression

- Squared Loss (Mean Squared Error)

$$\ell(h(x), y) = (h(x) - y)^2$$

- Absolute Loss

$$\ell(h(x), y) = |h(x) - y|$$

Supervised Learning Setup

Input: A finite data set generated from an unknown distribution $(x, y) \sim \mathcal{D}$ over inputs and labels, and a hypothesis class H

The data set is assumed to be sampled **independently and identically distributed** from some underlying distribution $\mathcal{D}$.

Optimality: a hypothesis that minimizes the expected loss:

$$h^\star = \arg\min_{h \in H} \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\ell(h(x), y)\right]$$

Empirical Objective: Since the true distribution $\mathcal{D}$ is unknown, we approximate this objective using the empirical loss:

$$\hat{h} = \arg\min_{h \in H} \frac{1}{m} \sum_{i=1}^{m} \ell(h(x_i), y_i)$$

Supervised Learning Setup

In practice, we split the available dataset into three parts:

Training set S_{train}:

- used to **learn the model parameters**
- optimize empirical risk:

Validation (evaluation) set S_{val}:

- used to **tune hyperparameters** (e.g., model size)
- helps detect **overfitting**

Test set S_{test}:

- used **only once at the end**
- provides an **unbiased estimate of generalization performance**

Never use test data during training or model selection.

Supervised Learning Algorithms

There are several approaches to finding (or approximating) $\hat{h}$:

- **Analytical solutions** Closed-form minimization (e.g., linear regression with squared loss)
- **Iterative optimization** Gradient-based methods (e.g., SGD) for optimizing differentiable models, and search-based or gradient-free methods for non-differentiable models
- **Structured / symbolic methods** Search over hypothesis space (e.g., decision trees, rule learning)
- **Kernel / instance-based methods** Implicit hypothesis spaces (e.g., SVMs, nearest neighbors)
- **Pretrained / foundation models** Start from a large pretrained model and adapt:
  - fine-tuning on S
  - prompting / in-context learning

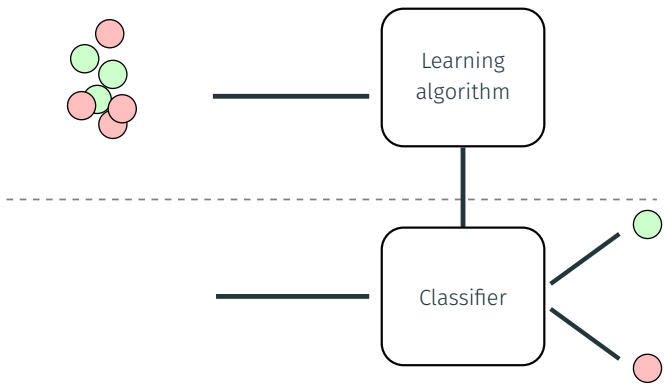
Example Approach: Iterative Supervised Learning via ERM

Algorithm 1 Iterative Supervised Learning via ERM

Require: Training set $S = \{(x_i, y_i)\}_{i=1}^{m}$, hypothesis class H, loss ℓ

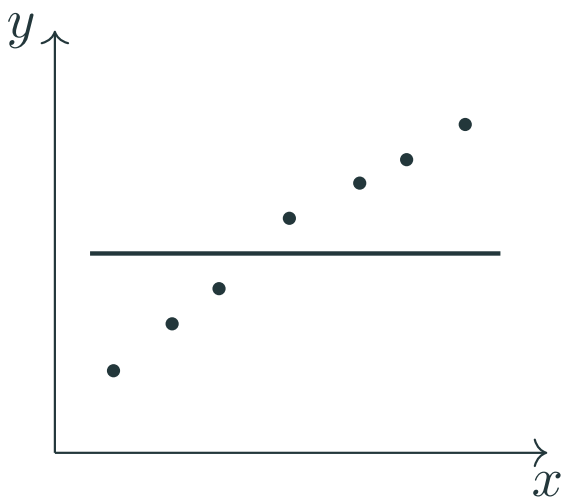
Ensure: Learned hypothesis $\hat{h}$

1: Initialize hypothesis $h \in H$
2: **repeat**
3: Update h using optimization (e.g., gradient descent)
4: **until** convergence
5: $\hat{h} \leftarrow h$
6: **Return** $\hat{h}$



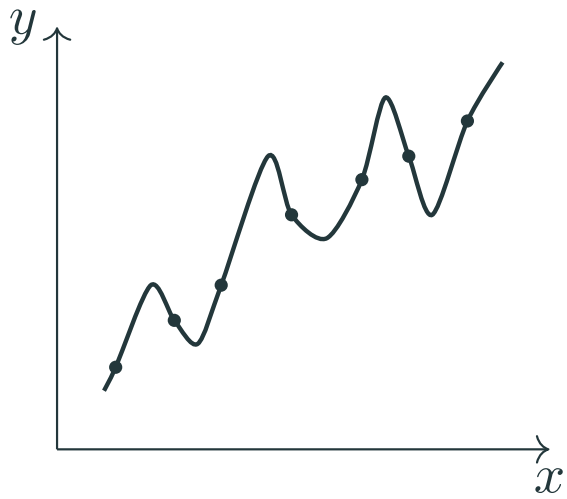
Supervised Learning : Key challenges

Underfitting



Avoid **underfitting**: using a model that is too simple to capture patterns.

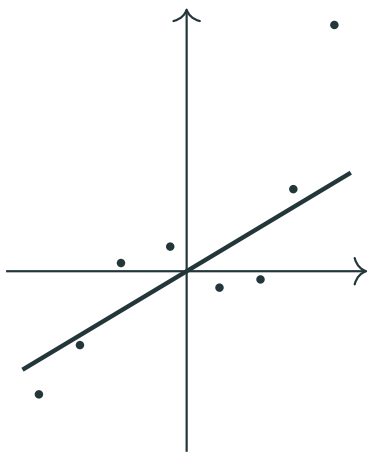
Overfitting



Avoid **overfitting**: fitting the training data too closely while failing to generalize.

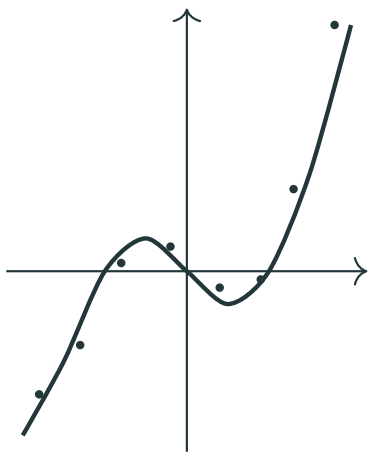
Example: Choosing Model Complexity

Assume the data is generated from an unknown cubic relationship: $y = x^3 - x + \epsilon$ where ϵ is a small noise term.



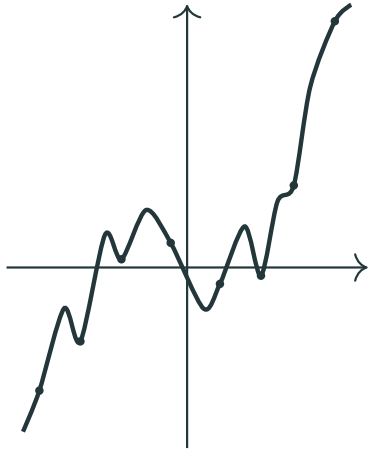
Fit with a **linear model**:

Too simple to capture the cubic trend.



Fit with a **degree-3 polynomial**:

Captures the underlying structure well.



Fit with a **n-degree polynomial**:

Fits the noise rather than the true pattern.

The goal is to find a model complexity that **generalizes well**.

Supervised Learning: Then vs. Now

Classical Supervised Learning

- Fixed dataset $S = \{(x_i, y_i)\}_{i=1}^{m}$
- Labels provided by humans or trusted sources
- Limited by the size and cost of labeled data
- Goal: learn $h \in H$ that generalizes to $\mathcal{D}$

Modern Setting with Foundation Models

- Large-scale pretrained models provide representations
- Unlabeled data can be **automatically labeled** (pseudo-labels)
- Data can be **generated or augmented** by models

Key Shift: We move from learning from labeled data to learning from data labeled by models. However, labels may be **noisy or biased**

Supervised Learning as World-Model Learning

In many AI systems, supervised learning is used to learn and improve a model of the world.

World models learn predictive representations of the environment, modeling how observations, states, or latent variables evolve over time.

They range from deterministic predictors to probabilistic models (e.g., state-space and latent variable models) that capture uncertainty and support planning and reinforcement learning.

Supervised Learning as World-Model Learning

World models can be designed to support different types of predictions:

- predict object categories from sensor observations
- predict the next state given the current state and action (i.e., learn system dynamics)
- predict system failures from measurements
- infer hidden or latent properties from observations

These predictive models can be used to support planning and decision-making.

Foundation Models as Policies

Recent AI systems increasingly use **foundation models directly as decision-making policies**.

Instead of only predicting the world, these models can map observations and instructions directly to actions.

Examples include:

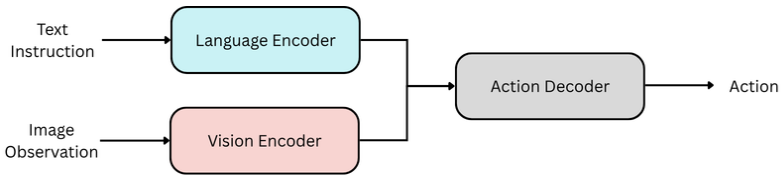
- Vision-Language-Action (VLA) models for robotics
- Large Language Models (LLMs) used as agents
- Multimodal embodied agents

Foundation Models as Policies

VLA models combine:

- **Vision** - understanding sensor observations
- **Language** - interpreting goals and instructions
- **Action** - generating control commands

From wikipedia:



These models are often trained from large datasets of demonstrations: $(observation_t, instruction_t) \rightarrow action_t$

This paradigm is closely related to imitation learning and behavioral cloning.

From Supervised Learning to Decision-Making

How does supervised learning relate to decision-making?

From Supervised Learning to Decision-Making

How does supervised learning relate to decision-making?

Supervised learning assumes a **fixed dataset**, while decision-making involves **interaction** with the environment.

We will explore two key connections:

- **Behavior cloning (imitation learning):** learn a policy from **expert demonstrations**, treating actions as labels.
- **Active learning:** **select** which data to acquire.

Behavior Cloning

Behavioral Cloning

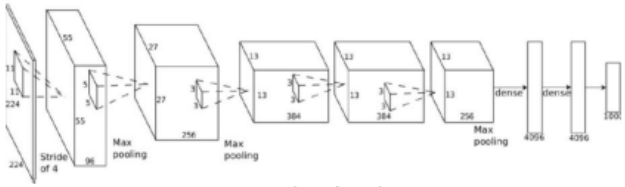
Imitation learning is a family of methods where an agent learns to act by observing an expert.

Behavior cloning is the simplest form of imitation learning, where the objective is to learn a policy by mimicking the behavior demonstrated by an expert.

A policy is trained to minimize the discrepancy between its predicted actions and the expert's actions over the collected dataset.

Behavior Cloning: Example

$\mathbf{o}_t$

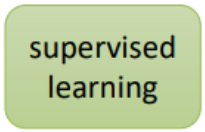


$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

$\mathbf{a}_t$

$\mathbf{o}_t$

$\mathbf{a}_t$



$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

Behavioral cloning

Can be broken down into the following steps:

- **Data Collection:** Gather a dataset of state-action pairs (s_i, a_i), where s_i represents the state and a_i represents the action taken by the expert at that state.
- **Model Selection:** Choose a model to approximate the expert's policy. Typically, a neural network $\pi_\theta(s)$ parameterized by θ.

Ideas for managing the learning process?

Behavioral Cloning

Loss: measures the difference between the expert's actions and the actions predicted by the model. For example, mean squared error (MSE) or cross-entropy loss for discrete actions leads to the following empirical risk:

$$\hat{\mathcal{R}}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \|a_i - \pi_\theta(s_i)\|^2$$

where N is the number of state-action pairs in the dataset.

Training: Optimize the model parameters θ by minimizing the empirical risk (e.g., using gradient descent)

$$\theta^* = \arg\min_\theta \hat{\mathcal{R}}(\theta)$$

Policy Execution: Use the trained model $\pi_{\theta^*}(s)$ as the policy to predict actions for new states during execution.

Strengths? Limitations?

Strengths of Behavioral Cloning

- **Simple and intuitive:** reduces policy learning to standard supervised learning
- **Data-driven:** can learn directly from expert demonstrations without specifying a reward function
- **Efficient training:** can leverage existing optimization methods and large supervised datasets
- **Useful when expert behavior is available:** especially effective when demonstrations are abundant and high quality
- **Good initialization:** often provides a strong starting point for further policy improvement

Behavior cloning makes sequential decision-making accessible through standard supervised learning tools.

Supervised Learning Assumption

We learn a hypothesis by minimizing empirical risk:

$$\hat{h} = \arg\min_{h \in H} \frac{1}{m} \sum_{i=1}^{m} \ell(h(x_i), y_i)$$

Key assumption: data is sampled i.i.d. from a fixed distribution

$$(x, y) \sim \mathcal{D}_{\text{train}} = \mathcal{D}_{\text{test}}$$

Implication
Generalization guarantees hold only under this assumption.

Limitations of Behavioral Cloning

In many settings, the IID assumption breaks because:

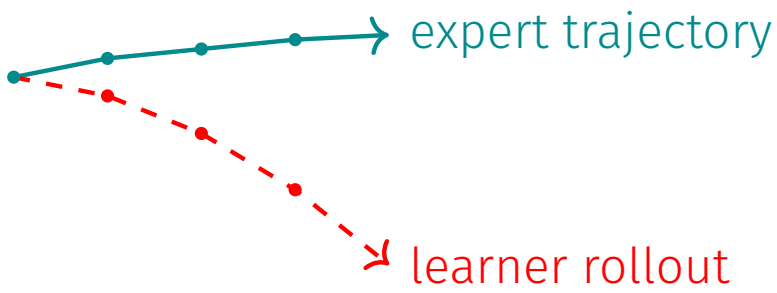
- Decisions are sequential: predictions are used to make decisions, which affect future inputs.
- The data distribution changes, i.e.,

$$\mathcal{D}_{\text{train}} \neq \mathcal{D}_{\text{test}}$$

The difference between the training and test distributions is called: **distribution shift**

- Train on one distribution, deploy on another
- Model encounters inputs it has never seen before
- Generalization guarantees no longer apply

Limitations of Behavioral Cloning



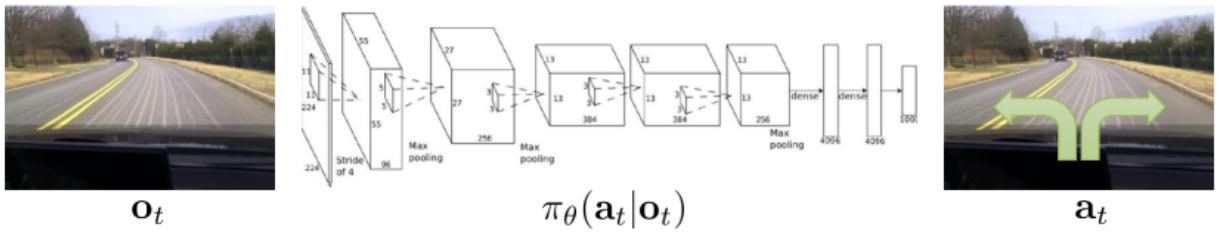
Distribution Shift: The agent follows π_θ trained on (s, a) from expert demonstrations, but encounters states that were never demonstrated.

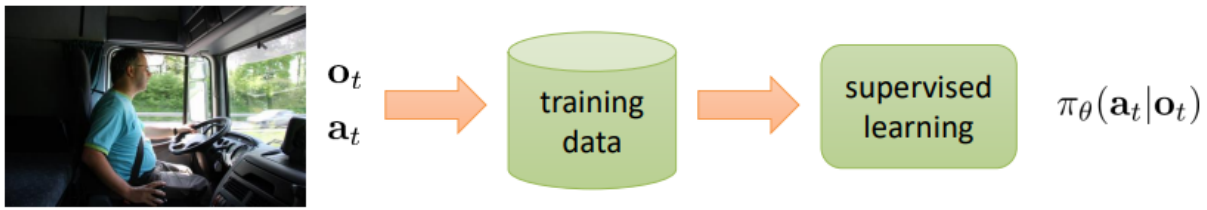
Key challenge: Errors compound over time.

- Small prediction error leads to a slightly wrong action
- The system transitions to a new, unseen state
- Errors increase as the agent moves further from expert behavior

Effect: performance degrades rapidly.

Limitations of Behavioral Cloning





In autonomous driving, a small steering mistake moves the car off the lane, a situation rarely observed in expert data.

DAgger: Addressing the Distribution Shift

Key idea: train on the states the learner actually visits

- Run current policy π_i to collect states $s \sim \mathcal{D}^{\pi_i}$
- Query expert for correct actions a^*
- Aggregate dataset:

$$D \leftarrow D \cup \{(s, a^*)\}$$

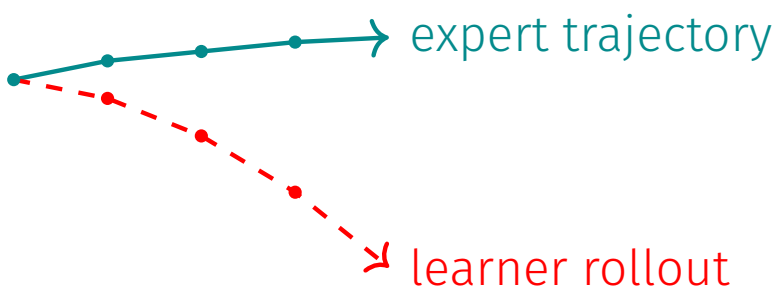
- Retrain policy on updated dataset

DAgger (Dataset Aggregation)

Algorithm 2 Dataset Aggregation (DAgger)

1: Initialize dataset $\mathcal{D} = \{(o_1, a_1), \ldots, (o_N, a_N)\}$
2: **for** iterations **do**
3: Train policy $\pi_\theta(a_t \mid o_t)$ on $\mathcal{D}$
4: Collect observations $\{o_1, \ldots, o_M\}$ using π_θ
5: Query expert for labels $\{a_1^*, \ldots, a_M^*\}$
6: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_1, a_1^*), \ldots, (o_M, a_M^*)\}$
7: **end for**

DAgger



DAgger queries the expert on states visited by the learner, and adds these corrections to the training set.

What assumptions are made here about access to the expert during training?

Limitations of Behavioral Cloning

Is this the same observation?

Limitations of Behavioral Cloning

Is this the same observation?

Limitations of Behavioral Cloning

Is this the same observation?

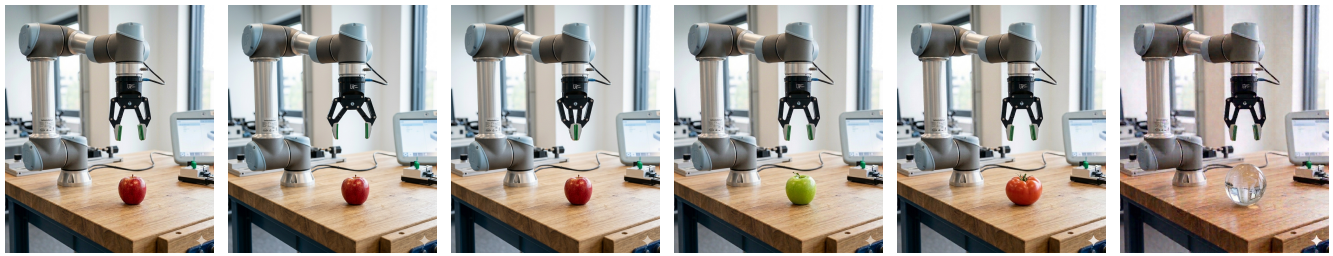
Limitations of Behavioral Cloning

Is this the same observation?

Limitations of Behavioral Cloning

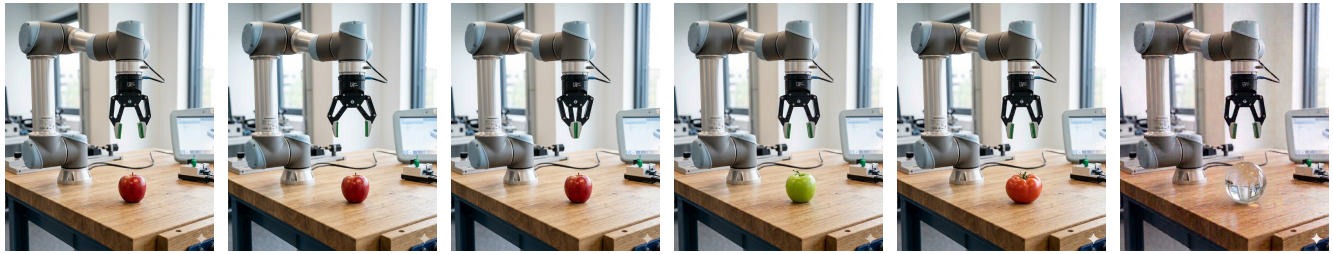
Is this the same observation?

Limitations of Behavioral Cloning



BC assumes a consistent mapping from observations to actions - this breaks under ambiguity and distribution shift.

Limitations of Behavioral Cloning



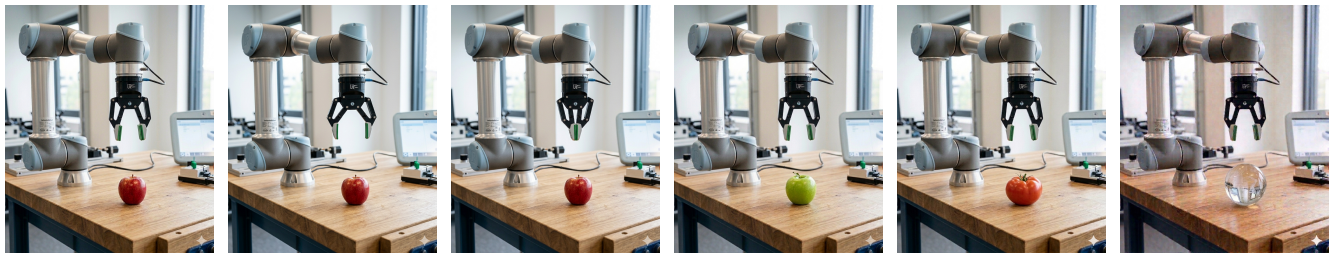
State aliasing \Perceptual aliasing (POMDP view): different underlying states produce visually similar observations, but require different actions:

$$o \approx o' \quad \text{but} \quad a^\star \neq a^{\star\prime}$$

In behavior cloning, the policy is learned as a direct mapping from observations to actions, i.e., $\pi(a \mid o)$.

In this case, conflicting supervision is recevied and classical methods tend to learn the **average action**: $\mathbb{E}[a \mid o]$ which can be suboptimal or even invalid.

Limitations of Behavioral Cloning



The opposite can also occur: The same underlying state may appear visually different, and therefore be treated as out-of-distribution.

Covariate shift \Observation variability: The same state appears differently across contexts:

$$s = s' \quad \text{but} \quad o \neq o'$$

→ *observations fall outside training distribution*

Behavior Cloning: Key Takeaways

Behavior Cloning works well when:

- Observations are **informative** (no state aliasing)
- The action distribution is **unimodal** (a single action per state)
- Train and test distributions are **aligned**

Behavior Cloning often fails with:

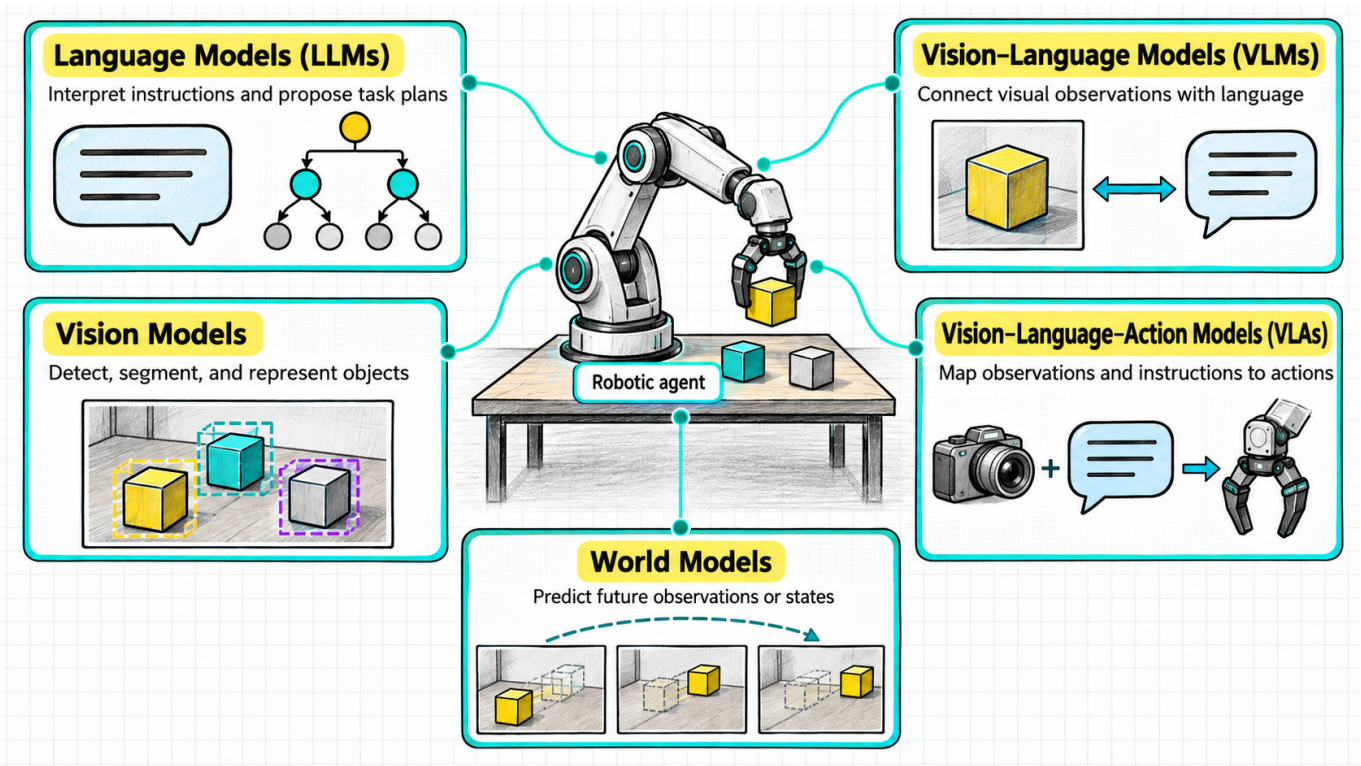
- **Distribution shift:** $\mathcal{D}_{\text{train}} \neq \mathcal{D}_{\text{test}}$
- **State aliasing:** $o \approx o'$ but $a^\star \neq a^{\star\prime}$
- **Multimodality:** $\pi(a \mid o)$ has multiple valid modes

Key insight: Behavior cloning reduces decision-making to supervised learning — but **sequential structure and partial observability break its assumptions.**

This motivates interactive and sequential methods (e.g., DAgger, RL, ...).

FM for Robotics

Foundation Models for Robotics



Robotics Solved?

Robotics Solved?

Still unresolved

Reliable multi-hour autonomy, guarantees under shift, dexterity in unstructured homes.

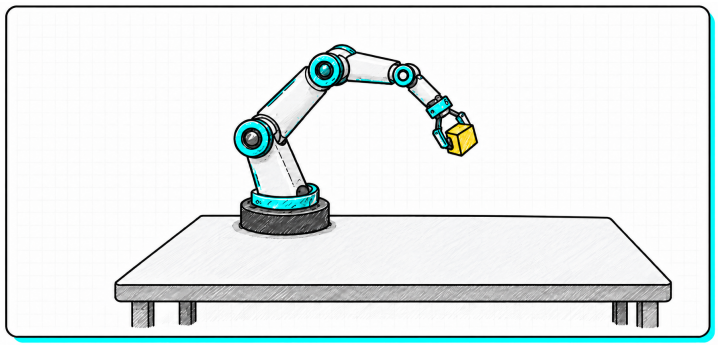
Persistent bottlenecks

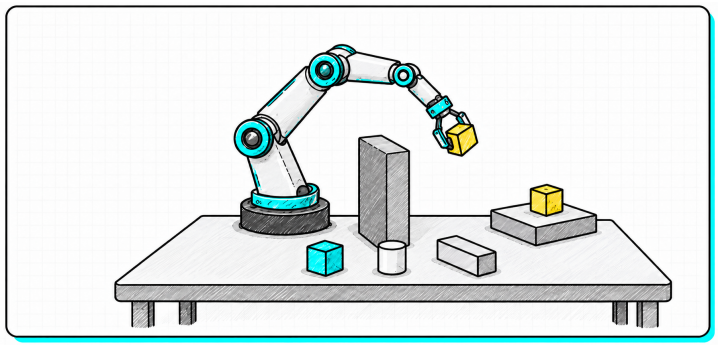
Long-horizon reliability, uncertainty, manipulation, safety and evaluation remain unresolved.

Data engines

Teleoperation, large-scale demonstrations, simulation and synthetic augmentation now shape progress.

Example





Vision–Language–Action

Foundation models as planners: Vision-language-action (VLA) policies connect semantic instructions and perception to robot actions.

maps: (image, instruction) $\rightarrow$ **motor actions**

Inputs: Camera / depth / proprioception Natural-language task Optional robot history

VLA Policy: $\pi(a_{t:t+H} \mid o_{\leq t}, g)$

Outputs:

End-effector poses Joint commands Discrete skill selections

Challenge:

language-level intent must translate into continuous, contact-rich, feasible motion.

Vision–Language–Action Models

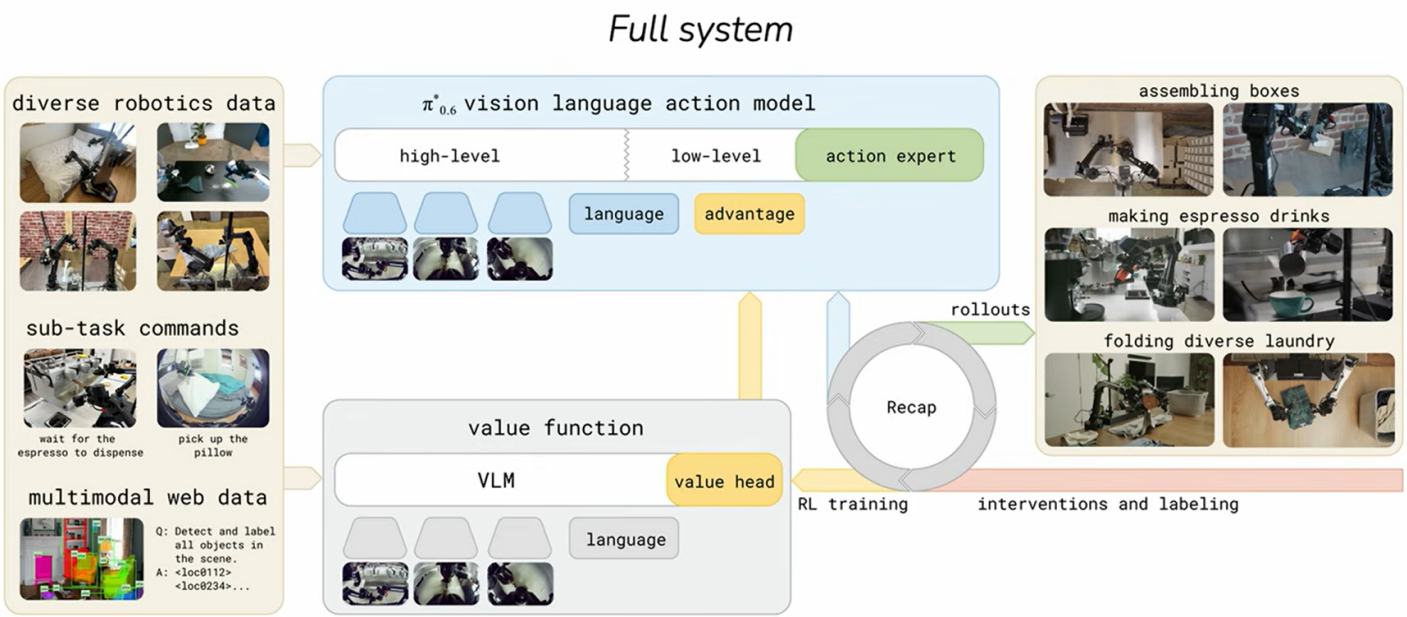
A **Vision–Language–Action (VLA) model** is a learned robot policy that connects visual observations and language instructions to physical actions.

- **Vision:** What does the robot observe?
  Camera images of the scene and objects.
- **Language:** What should the robot do?
  Example: “Place the red block in the basket.”
- **Action:** How should the robot move?
  Joint targets, end-effector motions, or gripper commands.

$$a_t \sim \pi_\theta(\cdot \mid o_t, \ell)$$

o_t: observation, potentially including robot state and history; ℓ: language instruction.

VLA Structure



How Does a VLA Represent Actions?

Example: an end-effector action

$$a_t = \big(\Delta x, \Delta y, \Delta z,\ \Delta\phi, \Delta\theta, \Delta\psi,\ g\big)$$

Translation, rotation, and a gripper command g. The action representation depends on the robot.

- **Discrete action tokens.**
  Quantize numerical action values into bins and predict their tokens using a language-model-style decoder.
  Example: the original OpenVLA.
- **Continuous action generation.**
  An action head generates continuous commands, often as an **action chunk**:

$$A_t = (a_t, \ldots, a_{t+H-1}).$$

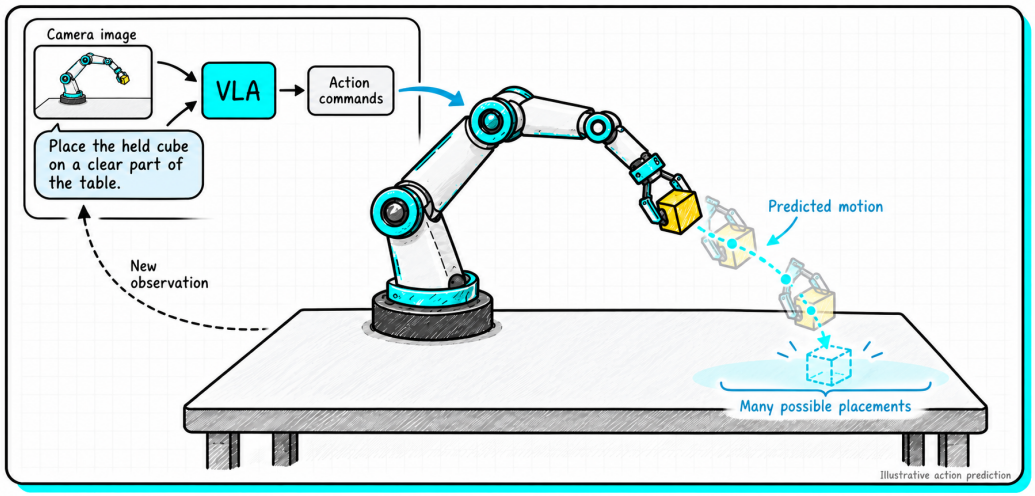
  Example: π_0 uses flow matching.

How Are VLAs Trained?

- **Start from a pretrained vision–language model.**
  Transfer visual and semantic representations learned from large image–text datasets.
- **Learn actions from robot demonstrations.**
  Train on observations, instructions, and demonstrated actions:
  $$\mathcal{D} = \{(o_t, \ell, A_t)\}.$$
  Behavior cloning learns to reproduce the demonstrated behavior.
- **Adapt to the target robot and tasks.**
  Fine-tune using data from the deployment setting.
- **Optionally improve through interaction.**
  Use execution outcomes and human corrections to refine the policy through reinforcement learning.

A VLA in an Uncluttered Setting

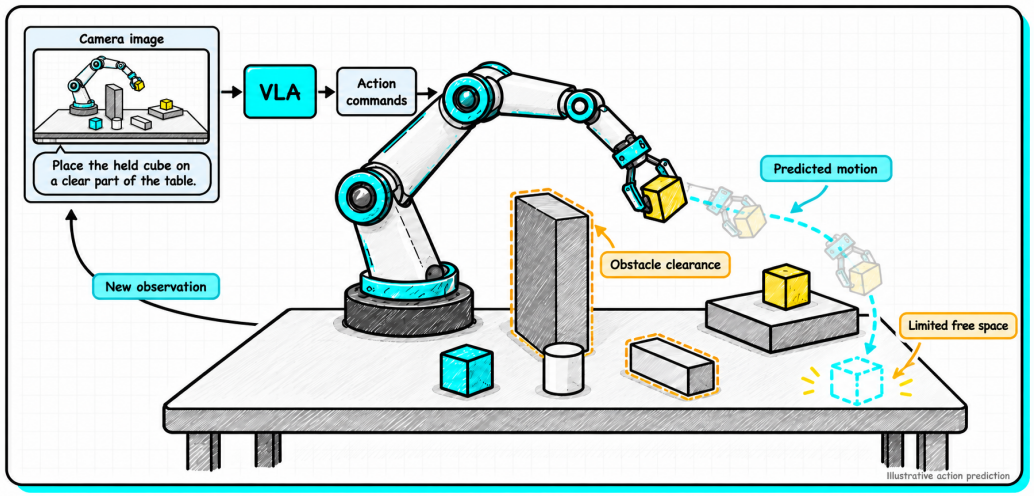
Instruction: “Place the cube on a clear part of the table.”



- **Input:** camera observation, instruction, and robot state if supported.
- **Output:** motion commands to approach the surface, place the cube, release, and retract.
- **Favorable geometry:** many possible placements and relatively large clearances.

The Same VLA in a Cluttered Setting

Same instruction: “Place the cube on a clear part of the table.”



- **Select a placement:** identify enough free space for the cube and the gripper.
- **Reach it safely:** account for obstacles along the motion of the arm, gripper, and held cube.
- **Complete the placement:** release and retract without disturbing nearby objects.

VLAs: The Same Instruction, Different Motion Constraints

Instruction: "Place the held cube on a clear part of the table."

Empty table

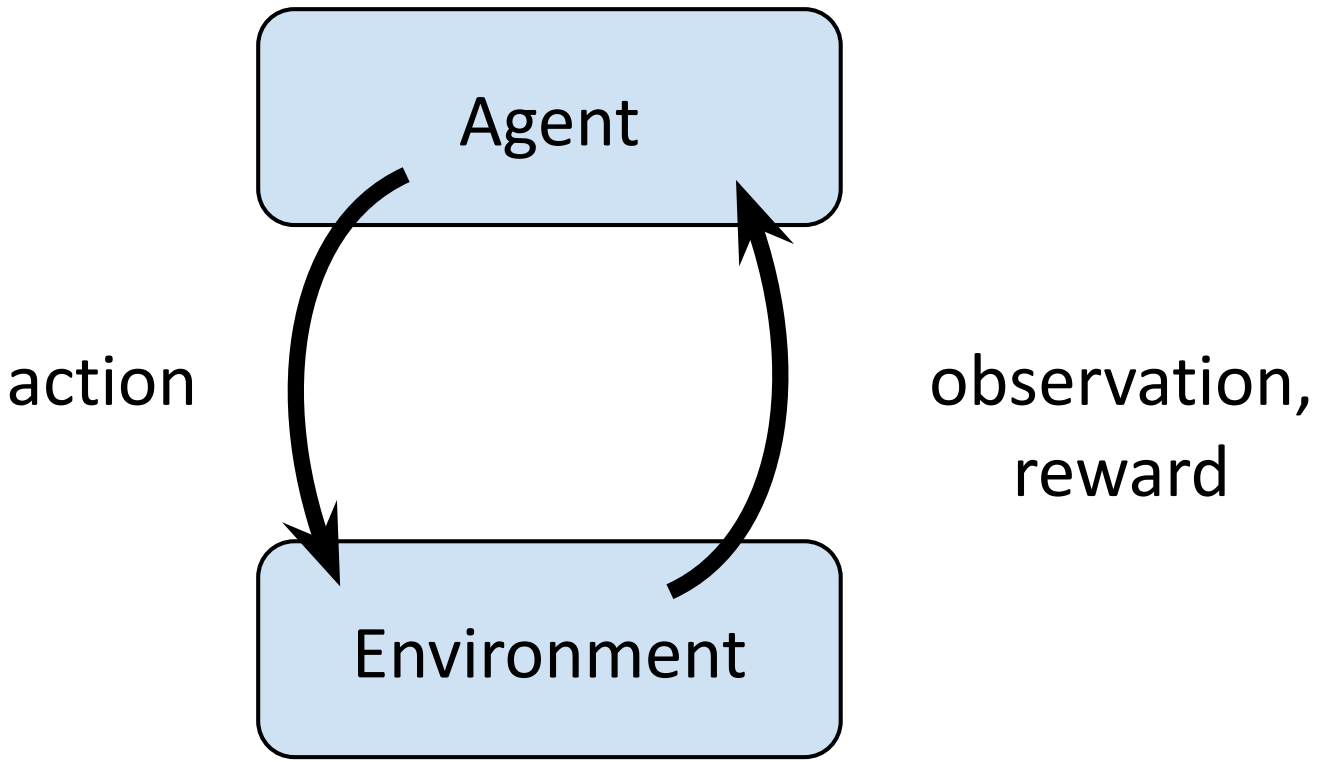
- Many valid placement locations.
- Large clearances for motion.
- Greater tolerance for prediction errors.

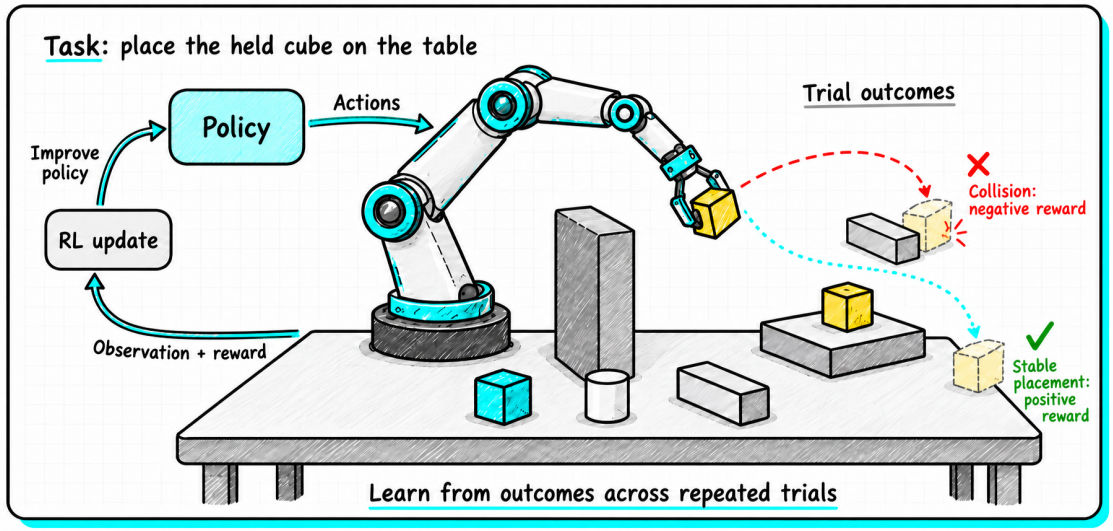
Cluttered table

- Infer obstacle geometry and free space.
- Avoid collisions during placement and retraction.
- Small errors can disturb nearby objects.

The challenge for a VLA: adapt its learned observation-to-action mapping to the exact scene, including object arrangements absent from training.

RL for Robotics


Agent
action
observation,
reward
Environment


Task: place the held cube on the table
Policy
Actions
Improve policy
RL update
Observation + reward
Trial outcomes
Collision: negative reward
Stable placement: positive reward
Learn from outcomes across repeated trials

Reinforcement Learning: Learning from Interaction

An agent learns a **policy** through interaction with an environment, using rewards to evaluate its behavior.

- **Observe and act.**
  The agent receives an observation and selects an action according to its policy.
- **Receive feedback.**
  The environment changes and provides a reward.
- **Improve the policy.**
  Use experience to increase expected cumulative reward:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right].$$

Compared with supervised learning: feedback evaluates the consequences of actions, rather than specifying the correct action at every step.

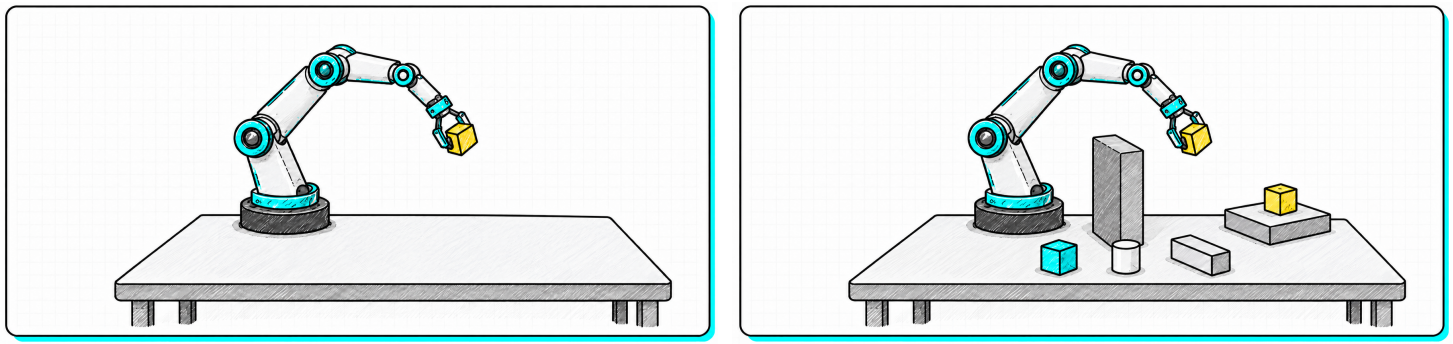
Reinforcement Learning in Robotics

RL can help robots learn behaviors whose success depends on **sequences of physical interactions**.

- **What can be learned?**
  Locomotion, grasping, contact-rich manipulation, and recovery from execution errors.
- **What makes it challenging?**
  Physical trials take time, exploration can damage hardware, and rewards may arrive only after a long action sequence.
- **How can training become practical?**
  Start from demonstrations or pretrained policies, reuse collected experience, and train in simulation.
- **What remains at deployment?**
  Handle sensing uncertainty and differences between training conditions and the real robot.

Example: Improving a Placement Policy with RL

Task: Place the held cube on the table without disturbing other objects.



Example: Improving a Placement Policy with RL

1. **Initialize from demonstrations.**
   Train a policy, potentially a VLA, to imitate successful placement motions.
2. **Collect interaction experience.**
   Execute the policy on varied object arrangements, initially in simulation or under supervised conditions.
3. **Evaluate the outcomes.**
   Reward stable placement; penalize collisions, dropped cubes, and unnecessary motion.
4. **Update and repeat.**
   Apply an RL algorithm to improve expected return, then collect experience with the updated policy.

Demonstrations provide an initial behavior; RL uses its consequences to improve it.

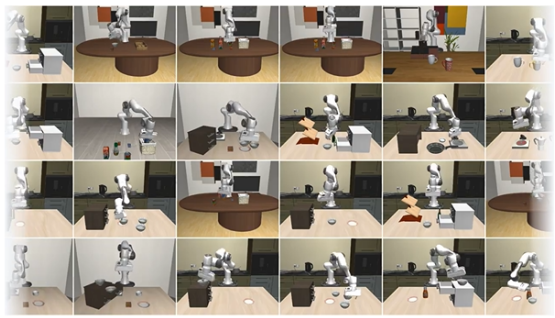
Conclusion

Why do we still need to understand robotics?

Why do we still need to understand robotics?

What are included

- ✓ RGB images from workspace and wrist cameras
- ✓ Proprioception
- ✓ Language task specifications
- ✓ PDDL scene descriptions



Layered Decision-Making